

Jenny Guerrero (jxg943), Madeleine Clore (mfc56), Meg Gopu (mxg724)

Table of contents

1. How to reproduce the project (page 2)
2. Description of the database (pages 3-5)
 - a. ER diagram (page 3)
 - b. Table explanation (page 3-5)
3. Use cases (pages 6-21)
 - a. Add new book to library (page 6-8) - Meg
 - b. Return book to library (pages 9-10) - Meg
 - c. Remove author and all books (pages 11-12) - Jenny
 - d. Update author name (pages 13-15) - Jenny
 - e. Check out a book (pages 16-18) - Maddie
 - f. Update a member's status from active to inactive (pages 19-20) - Maddie
 - g. Use cases 7-12 (pages 21)
4. Physical design (page 22)
5. User manual (page 23)
6. Reflection (page 23)

1. How to reproduce

Name of database backup file:

Name of zip file that contains Java code: TestProject.zip

Demo date: 12/8/23 1:30-2 pm TA: Debargha Ganguly

Virtual environment: Meghana Gopu

SQL Server access:

Name: Meghana Gopu

Case ID: mxg724

SA password: |*jYKO6&m^aP@m

SQL Server name: exp-sql-03

Database name: Library

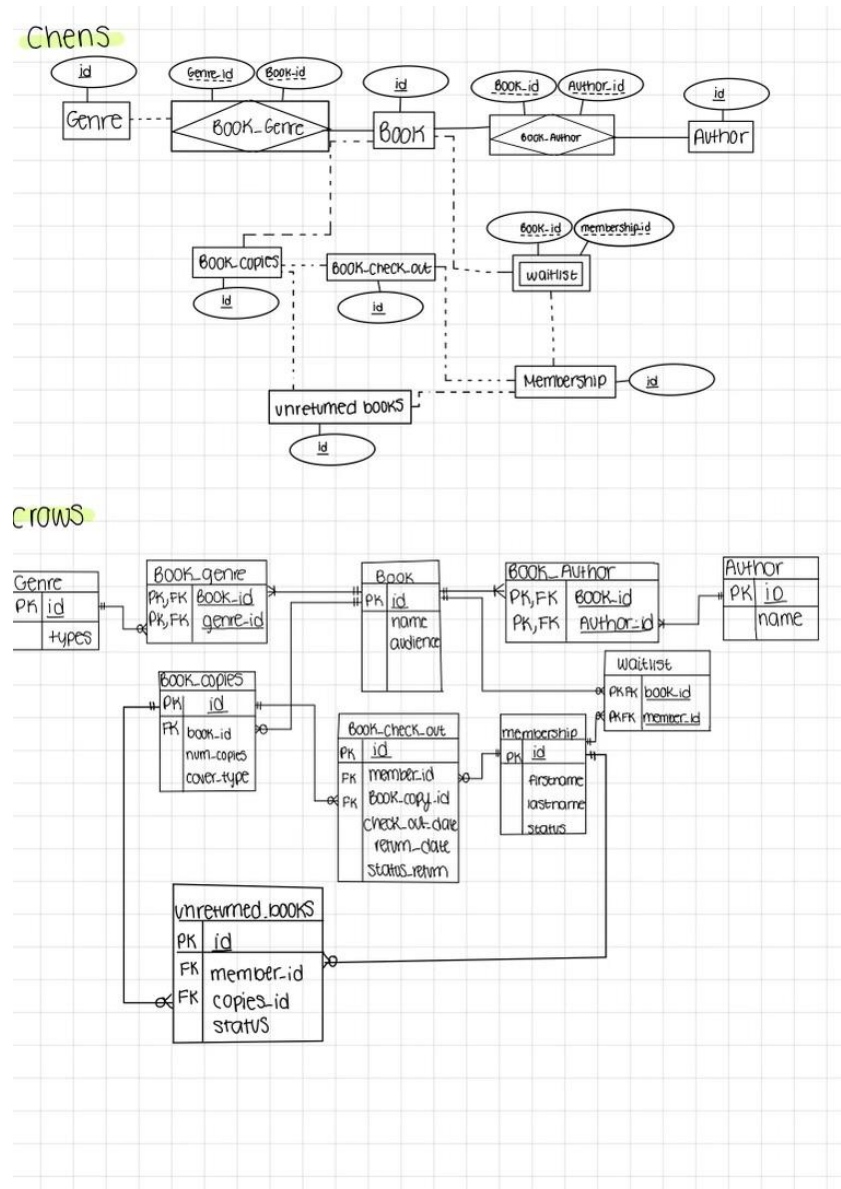
Database user name: bookUser

Database Password: Database341

2. Description of database

Our database will keep track of books in a library and their authors. It will keep track of how many copies of the book the library has, as well as members who have/want to check out a book. The UI will allow a user to insert and delete books to the database and the database will be updated appropriately. The UI will also allow the user to run pre-defined queries and see their output.

a. ER diagram



b. Table explanation

In our Library database, we include the following tables and attributes:

Author(author_id, name)
 Book(book_id, name, audience)
 Membership(member_id, firstname, lastname, status)
 Genre(genre_id, genre_type)
 Book_Author(book_id, author_id)
 Book_Genre(book_id, genre_id)
 Waitlist(book_id, member_id)
 Unreturned_Books(id, member_id, copies_id, status)
 Book_Copies(copies_id, book_id, num_copies, cover_type)
 Book_Check_Out(check_out_id, number_id, copies_id, check_out_date, return_date, status_return)

The Author table includes an author_id which is a primary key assigned to each author that has a book in the library and the name of the author. We allow for books to have multiple authors and for authors to write multiple books by including a Book_Author table.

The Book table includes a book_id which is the primary key for each book in the library, along with the name of the book and the target audience of the book. Every book has a different ID and only appears in the table once. If there are multiple copies of a book, then the ID for that book will appear in the Book_Copies table with a different copies_id. We are assuming that each book in the library only has one target audience. Each book also has several genres, which are found in the Genre table. We connect the tables with the Book_Genre table because books can have multiple genres and the same genre can belong to multiple books.

The Membership table includes the member_id, along with the first name, last name, and status of each member belonging to the library. The status is whether the member is active or inactive. Only members are allowed to check out books and get added to the waitlist for books. When a member checks out a book, the return date is automatically set for 2 weeks after the checkout date. This information is kept track of in the Book_Check_Out table which. When members check out a book that they were waiting for, they get removed from the Waitlist table. After they return a book, they are not removed from the Book_Check_Out table.

The Genre table includes the genre_id and the genre_type (the genre of a book). When a genre is inserted, it is automatically assigned a genre_id, therefore it is

uniquely identified by the primary key, genre_id. The genre_type is assigned by the user.

Book_Author is a linking table between Book and Author since a book is allowed to have more than one author and an author is allowed to have more than one book. Book_Author holds the books corresponding to an author by storing the book_id with the author_id, therefore its primary key is a composite key of (book_id, author_id). Both book_id and author_id are set to be automatically deleted or updated depending on the corresponding changes made in Author and Book.

The Book_Genre table is a linking table between Book and Genre since a book is allowed to have more than one genre and a genre is allowed to be assigned to more than one book. Book_Genre holds the books corresponding to a genre by storing the book_id with the genre_id, therefore its primary key is a composite key of (book_id, genre_id). book_id is set to be automatically deleted depending on a corresponding deletion made in Book

The Waitlist table includes the book_id and member_id as the primary keys and the foreign keys. Members get added to the waitlist if they want to check out a book but all of the copies are in use. The book_id is assigned instead of copies_id because whichever copy of a book is first available will be assigned to them to check out.

The Unreturned_Book Table holds the unreturned books. It has foreign keys copies_id and members_id which reference the Membership table and the Book_Copies table.

The Book_Copies keeps track of how many copies of a single book the library has. Copies_id is the primary key and book_id is the foreign key which references Book.

Book_Check_Out keeps track of what books have been checked out by each member. It includes the check_out_id as the primary key of the relation that is unique for each check-out transaction, a member_id (references the id of the member from the Member table), the copies_id (references the id of the book copy from the Book_Copies table), the date that the book was checked out, the date that the book is expected to be returned, and the status of the book as either being 'returned' or 'not returned'. The member_id is set to be deleted or updated depending on the corresponding changes in the Membership table.

Use Case 1: Insert New Book in Series or a new Author.

There are always new up and coming authors or authors who write more than one book such as sequels. We want to add these new books or authors into the library and see what other books are in the library that the author wrote, if any. This will require us to insert the new book into the Book table and or author into the Author table and join them together in the Book_Author Table. So, this requires 3 inserts. Afterwards we want to see if the author has written other books in the library, so we select the statements that join the Book and Author table together, so one selected statement is needed. The only books that can be inserted into the table have to have a genre that is already in the database. This will require a Transaction, Rollback, and commit, so that it makes sure that if the inserted book has a genre_id that is not in the genre table it will rollback.

SQL Code:

1. Create a view with book name, author name, and genre type,

```
CREATE INDEX idx_Book_name ON Book (name);
CREATE INDEX idx_Author_name ON Author(name);
```

5. Since we want to insert a new book and/or new author while retrieving any other books the author has written, we will create a procedure to handle that.

```
CREATE PROCEDURE InsertNewBookProcedure3
    @book_name VARCHAR(50),
    @author_name VARCHAR(20),
    @genre_type VARCHAR(10)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
    DECLARE @author_id INT;
    DECLARE @book_id INT;
    DECLARE @genre_id INT;
    -- Check if the genre exists
    SELECT @genre_id = genre_id
    FROM Genre
    WHERE genre_type = @genre_type;
    -- If the genre doesn't exist, rollback the transaction
    IF @genre_id IS NULL
    BEGIN
```

```

        ROLLBACK; -- Roll back if the genre doesn't exist
        PRINT 'Genre does not exist. Transaction rolled back.';
        RETURN
    END
    ELSE
    BEGIN
        -- Check if the author exists
        IF NOT EXISTS (SELECT 1 FROM Author WHERE name = @author_name)
        BEGIN
            -- Insert the author if not already present
            INSERT INTO Author (name) VALUES (@author_name);
        END
        -- Check if the Book exists
        IF NOT EXISTS (SELECT 1 FROM Book WHERE name = @book_name)
        BEGIN
            -- Insert the book if not already present
            INSERT INTO Book (name) VALUES (@book_name);
        END
        -- Get the author ID
        SELECT TOP 1 @author_id = author_id
        FROM Author
        WHERE name = @author_name;
        -- Get the book ID
        SELECT TOP 1 @book_id = book_id
        FROM Book
        WHERE name = @book_name;
        IF NOT EXISTS (SELECT 1 FROM Book_Author WHERE book_id = @book_id AND
author_id = @author_id)
        BEGIN
            -- Insert the book_author if not already present
            INSERT INTO Book_Author(book_id, author_id) VALUES (@book_id, @author_id);
        END
        SELECT Book.name AS BookName
        FROM Book
        INNER JOIN Book_Author ON Book_Author.book_id = Book.book_id
        INNER JOIN Author ON Author.author_id = Book_Author.author_id
        WHERE Author.author_id = @author_id;
        COMMIT;
    END;
END;
```

7. Grant Permissions.

```
GRANT EXECUTE ON OBJECT :: InsertNewBookProcedure3 TO bookUser;  
GRANT SELECT ON Book TO bookUser;  
GRANT INSERT ON Book TO bookUser;  
GRANT UPDATE ON Book TO bookUser;  
GRANT DELETE ON Book TO bookUser;
```

```
GRANT SELECT ON Author TO bookUser;  
GRANT INSERT ON Author TO bookUser;  
GRANT UPDATE ON Author TO bookUser;  
GRANT DELETE ON Author TO bookUser;
```

```
GRANT SELECT ON Insert_New_Book TO bookUser;  
GRANT INSERT ON Insert_New_Book TO bookUser;  
GRANT UPDATE ON Insert_New_Book TO bookUser;  
GRANT DELETE ON Insert_New_Book TO bookUser;
```

8. Create a user and login for the JDBC

```
CREATE LOGIN bookUser  
WITH PASSWORD = 'Database341',  
DEFAULT_DATABASE = Library,  
CHECK_POLICY = ON,  
CHECK_EXPIRATION = OFF ;  
create user bookUser  
from login bookUser;
```

Use Case 2: Book has been returned.

Members who have checked out a book, but they have not returned it to the library yet. When the Member finally returns the book, the librarian has to change the status of the book from not returned to returned. That is what my code will be doing. First we have to keep check of all the books that were not returned so we create a table, then we find out the books not yet returned so we use a select statement that will join the tables to find out the non returned books. Then we insert those values into another table using Insert statements. Then when the member finally returned the book we used a delete statement to delete the book in the not returned table and an update statement to update the status of the book in the book check out table.

1. Create a table that will hold the not returned books.

```
CREATE TABLE Unreturned_Books (
    unreturned_id INT IDENTITY,
    member_id INT,
    copies_id INT,
    status_return VARCHAR(15) DEFAULT 'not returned',
    PRIMARY KEY (unreturned_id),
    FOREIGN KEY (member_id) REFERENCES Membership (member_id) ON DELETE
    CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (copies_id) REFERENCES Book_Copies (copies_id)
);
```

2. Insert the not return book values into this table.

```
INSERT INTO Unreturned_Books (member_id, copies_id)
SELECT member_id, copies_id
FROM Book_Check_Out
WHERE status_return = 'not returned';
```

3. Create an index

```
CREATE INDEX idx_Book_Check_Out_Member_Copies ON
Book_Check_Out(member_id, copies_id);
```

4. Create a procedure that will allow the librarian to update the return status of the book.

```
CREATE PROCEDURE UpdateBookStatusToReturned
    @memberID INT,
```

```

    @copiesID INT
AS
BEGIN
BEGIN TRANSACTION;
    UPDATE Book_Check_Out
    SET status_return = 'returned'
    WHERE member_id = @memberID AND copies_id = @copiesID;

    -- Update Unreturned_Books table if the book status is updated
    DELETE FROM Unreturned_Books
    WHERE member_id = @memberID AND copies_id = @copiesID;
    COMMIT;
END;

```

5. Execute and Grant permissions.

```

GRANT EXECUTE ON OBJECT :: UpdateBookStatusToReturned TO bookUser;
GRANT SELECT ON Book_Check_Out TO bookUser;
GRANT INSERT ON Book_Check_Out TO bookUser;
GRANT UPDATE ON Book_Check_Out TO bookUser;
GRANT DELETE ON Book_Check_Out TO bookUser;

GRANT SELECT ON Unreturned_Books TO bookUser;
GRANT INSERT ON Unreturned_Books TO bookUser;
GRANT UPDATE ON Unreturned_Books TO bookUser;
GRANT DELETE ON Unreturned_Books TO bookUser;

```

Use case 3: Removing an Author and their individual Books

It's important to keep a library database updated and current. If there is a case where an author is controversial or the author's work contains controversial/offensive/inappropriate content then a librarian could choose to remove the author and their work from the library. The author will no longer be in the system, and the books solely by that author will be gone. The stored procedure takes care of this author deletion.

1. Create an index

```
CREATE INDEX idx_Author_name_Replace ON Author(name);
```

2. Create a stored procedure

```
CREATE PROCEDURE RemoveAuthorAndBooks
--declare the input variable
    @authorName VARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION

    -- variable to store author id that will be deleted
    DECLARE @authorId INT;
    -- assign the author id for author that will be deleted based on author name
    SELECT @authorId = author_id
    FROM Author
    WHERE name = @authorName;
    -- if there is not one matching author with the input name
    IF NOT EXISTS (SELECT 1 FROM Author WHERE name = @authorName)
    BEGIN
        -- then the author doesn't exist, raise an error and rollback the transaction
        RAISERROR('Author does not exist in the library.', 16, 1);
        ROLLBACK TRANSACTION;
        RETURN;
    END;

    -- but if the author does exist then
    IF @authorId IS NOT NULL
    BEGIN
        -- retrieve all info about the books associated with the author, useful for previewing
        SELECT * FROM Book
        -- select distinct books from Book_Author table and only returns those with just that author
```

```

WHERE book_id IN (SELECT BA.book_id
FROM Book_Author BA
GROUP BY BA.book_id
HAVING COUNT(*) = 1)
AND
    book_id IN (select book_id FROM Book_Author WHERE author_id = @authorId);
-- delete the books found above from Book
DELETE FROM Book
WHERE book_id IN (SELECT BA.book_id
FROM Book_Author BA
GROUP BY BA.book_id
HAVING COUNT(*) = 1)
AND
    book_id IN (select book_id FROM Book_Author WHERE author_id = @authorId);
-- delete the author from linking table
DELETE FROM Book_Author
WHERE author_id = @authorId;
-- delete the author from the Author table
DELETE FROM Author
WHERE author_id = @authorId;
COMMIT
END
END;

```

3. Executing the Stored Procedure:

EXEC RemoveAuthorAndBooks

@authorName ='Author Name Gone' ;

4. Grant permissions

```

GRANT EXECUTE ON OBJECT :: RemoveAuthorAndBooks TO bookUser;
GRANT SELECT ON Book_Author TO bookUser;
GRANT INSERT ON Book_Author TO bookUser;
GRANT UPDATE ON Book_Author TO bookUser;
GRANT DELETE ON Book_Author TO bookUser;

```

```

GRANT SELECT ON Author TO bookUser;
GRANT INSERT ON Author TO bookUser;
GRANT UPDATE ON Author TO bookUser;
GRANT DELETE ON Author TO bookUser;

```

```

GRANT SELECT ON Book TO bookUser;
GRANT INSERT ON Book TO bookUser;
GRANT UPDATE ON Book TO bookUser;
GRANT DELETE ON Book TO bookUser;

```

Use Case 4: Replace an Old Author with a New Author

It's important to keep a library database updated and current. If there is a case where an author has legally changed their name, then the Author information should be updated to store the new name. Or, if there is a case where an author was discovered to have stolen all of their works from another author, then the books from the old author should be updated to reference the new author instead of the old one, and the old author and their information should be removed from the library's records. A view is created to show the effects of replacing an author.

1. Create an index

```
CREATE INDEX idx_Book_Author_author_id ON Book_Author(author_id);
```

2. Create a view showing the author ID, the name of the author and the books of the author

```
CREATE VIEW Author_Books_View1 AS
SELECT
    A.author_id,
    A.name AS author_name,
    B.name AS book_name
FROM
    Author A
JOIN
    Book_Author BA ON A.author_id = BA.author_id
JOIN
    Book B ON BA.book_id = B.book_id;
```

3. Create a procedure that replaces the old author with the new author in any relations that reference the old author and removes the old author if the new author exists

```
CREATE PROCEDURE ReplaceAuthorName2
    @oldAuthorName VARCHAR(20),
    @newAuthorName VARCHAR(20)

AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @oldAuthorId INT, @newAuthorId INT;

    -- Find the ID of the old author
```

```

SELECT @oldAuthorId = author_id
FROM Author
WHERE name = @oldAuthorName;

-- Check if the old author exists
IF @oldAuthorId IS NOT NULL
BEGIN
    -- Check if the new author already exists
    SELECT @newAuthorId = author_id
    FROM Author
    WHERE name = @newAuthorName;

    -- If new author doesn't exist, update the old author's name
    IF @newAuthorId IS NULL
    BEGIN
        UPDATE Author
        SET name = @newAuthorName
        WHERE author_id = @oldAuthorId;
    END
    ELSE
    BEGIN
        -- If the new author exists, update references to the new author and remove the old author
        UPDATE Book_Author
        SET author_id = @newAuthorId
        WHERE author_id = @oldAuthorId;

        -- Delete the old author entry
        DELETE FROM Author
        WHERE author_id = @oldAuthorId;
    END
END
END;

```

4. Executing the Stored Procedure

```
EXEC ReplaceAuthor
```

```
@oldAuthorName = 'Old Author Name'
```

```
@newAuthorName = 'New Author Name'
```

5. Grant Permissions

```
GRANT EXECUTE ON OBJECT :: ReplaceAuthorName2 TO bookUser;  
GRANT SELECT ON Book_Author TO bookUser;
```

```
GRANT INSERT ON Book_Author TO bookUser;  
GRANT UPDATE ON Book_Author TO bookUser;  
GRANT DELETE ON Book_Author TO bookUser;
```

```
GRANT SELECT ON Author TO bookUser;  
GRANT INSERT ON Author TO bookUser;  
GRANT UPDATE ON Author TO bookUser;  
GRANT DELETE ON Author TO bookUser;
```

Use case 5: Check out a book

The user wants to check out a book from the library. The user must be a member of the library in order to check out a book, or an error will be produced. If they are a valid member, then they will be added to the Book_Check_Out table. This use case also checks whether the member was on the Waitlist for that book. If they were on the waitlist, they would be removed. Transactions with rollbacks and commit were also included.

1. Stored procedure

```
CREATE PROCEDURE checkoutBook3
```

```
    @member_id INT,  
    @copies_id INT,  
    @check_out_date DATE,  
    @return_date DATE,  
    @status_return VARCHAR (15)
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    BEGIN TRANSACTION; -- Start the transaction
```

```
    DECLARE @isAuthorized INT;
```

```
    -- Check if the member is authorized to check out books
```

```
    SELECT @isAuthorized = COUNT(*)  
    FROM Membership  
    WHERE member_id = @member_id;
```

```
    IF @isAuthorized = 0
```

```
    BEGIN
```

```
        -- Member is not authorized, rollback the transaction and exit
```

```
        RAISERROR('Unauthorized member. Transaction rolled back.', 16, 1);
```

```
        ROLLBACK TRANSACTION;
```

```
        RETURN;
```

```
    END;
```

```
    -- Check if the copies_id exists in Book_Copies table and get the corresponding book_id
```

```
    DECLARE @book_id INT;  
    SELECT @book_id = book_id  
    FROM Book_Copies  
    WHERE copies_id = @copies_id;
```

```

IF @book_id IS NULL
BEGIN
    -- Provided copies_id doesn't exist, rollback the transaction and exit
    RAISERROR('Invalid copies ID. Transaction rolled back.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END;

-- Check if the member has a waitlist entry for the corresponding book_id
DECLARE @waitlistCount INT;

SELECT @waitlistCount = COUNT(*)
FROM Waitlist
WHERE member_id = @member_id AND book_id = @book_id;

-- Insert into Book_Check_Out table for the book checkout
INSERT INTO Book_Check_Out (member_id, copies_id, check_out_date, return_date,
status_return)
VALUES (@member_id, @copies_id, @check_out_date, @return_date, @status_return);

-- If member has an entry in the waitlist for the book, remove the entry
IF @waitlistCount > 0
BEGIN
    DELETE FROM Waitlist
    WHERE member_id = @member_id AND book_id = @book_id;
END;

    IF @status_return NOT IN ('returned','not returned')
BEGIN
    -- Provided copies_id doesn't exist, rollback the transaction and exit
    RAISERROR('Invalid Status. Transaction rolled back.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END;

COMMIT TRANSACTION; -- Commit the transaction

```

END;

2. Permissions

GRANT EXECUTE ON OBJECT :: checkoutBook3 TO bookUser;

GRANT SELECT ON Waitlist TO bookUser;

GRANT INSERT ON Waitlist TO bookUser;

GRANT UPDATE ON Waitlist TO bookUser;

GRANT DELETE ON Waitlist TO bookUser;

GRANT SELECT ON Book_Check_out TO bookUser;

GRANT INSERT ON Book_Check_out TO bookUser;

GRANT UPDATE ON Book_Check_out TO bookUser;

GRANT DELETE ON Book_Check_out TO bookUser;

Use case 6: Update a member's status from active to inactive

The user wants to change their status (either from active to inactive, or inactive to active). If they change their status to inactive, then all of the tuples in the Book_Check_Out table with that member ID will be removed. Changing the status back to active does not bring back the user's past check out history. A transaction with rollbacks and commit are included.

```

CREATE PROCEDURE updateMembership
    @member_id INT,
    @status VARCHAR(8)

AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;

    UPDATE Membership
    SET status = @status
    WHERE member_id = @member_id

    DECLARE @isAuthorized INT

    SELECT @isAuthorized = COUNT(*)
    FROM Membership
    WHERE member_id = @member_id
    IF @isAuthorized = 0
    BEGIN
        -- Member is not authorized, rollback the transaction and exit
        RAISERROR('Unauthorized member. Transaction rolled back.', 16, 1);
        ROLLBACK TRANSACTION;
        RETURN;
    END;
    DELETE FROM Book_Check_Out
    WHERE member_id = @member_id
    COMMIT TRANSACTION;

END;

GRANT EXECUTE ON OBJECT :: updateMembership TO bookUser;
GRANT SELECT ON Membership TO bookUser;

```

```
GRANT INSERT ON Membership TO bookUser;  
GRANT UPDATE ON Membership TO bookUser;  
GRANT DELETE ON Membership TO bookUser;
```

Use case 7: Find all books of a certain genre

The user wants to find all of the books the library owns of a specific genre. They also want to be able to select different target audiences so a different set of books will show up. This requires 1 select statement that joins Book, Genre, Book_Genre, Author, and Book_Author

Use case 8: Which books are available to check out

The user wants to check out a book immediately, so they look for all books that are available. This requires 1 select statement that joins Book_Check_Out, Book_Copies, and Book tables.

Use case 9: Remove a book from the library and from check out because the member lost a book

The member lost a book and told the library. This means that the library must remove the book copy from the library. This involves removing book copy from Book_Copies.

Use case 10: Check the history of book checkouts

The member wants to see all of the books and their genres that they have checked out since becoming a member of the library. This requires 1 select statement that joins Book_Check_Out, Book_Copies, and Book_Genre.

Use case 11: Which genre is the most popular

The user wants to determine which genre of book is the all-time most popular genre to check out in the library. This requires 1 select statement that joins the Book_Check_Out, Book_Copies, Book, and Book_Genre tables.

Use case 12: Which author wrote the most books and what books they wrote

The user wants to determine the author most featured in the library as well as the books that that author wrote. This requires 1 select statement that joins Author, Book_Author, and Book and keeps track of the number of books per author (group by) then sorts in descending order and returns the first (author with most books) entry.

4. Functional dependencies/physical database design/normalization

Author

F.D.: {author_id → name}

Book

F.D.: {book_id → name/audience}

Membership

F.D.: {member_id → firstname/lastname/status}

Genre

F.D.: {genre_id → genre}

Book_Author

F.D.: {book_id/author_id}

Book_Genre

F.D.: {book_id/genre_id}

Waitlist

F.D.: {book_id/member_id}

Book_Copies

F.D.: {copies_id → book_id/num_copies/cover_type}

Book_Check_out

F.D.: {check_out_id →
member_id/copies_id/check_out_date/return_date/status_return}

CREATE INDEX idx_Book_name ON Book (name);

CREATE INDEX idx_Author_name ON Author(name);

CREATE INDEX idx_Book_Check_Out_Member_Copies ON
Book_Check_Out(member_id, copies_id);

CREATE INDEX idx_Author_name_Replace ON Author(name);

CREATE INDEX idx_Book_Author_author_id ON Book_Author(author_id);

CREATE INDEX IX_Book_Copies_copies_id ON Book_Copies(copies_id);

CREATE INDEX IX_Waitlist_member_id_book_id ON Waitlist(member_id, book_id);

5. User manual

The User interface is implemented in Java. The user will input information in the front end Java code, allowing them to perform multiple actions. The information they will need is book name, author name, genre type, member id, copies id, check out date and return date, status of book, and member status. The front end will provide a menu, containing different options on what action the user wants to perform, and then it will ask the user specific inputs to actually execute the action.

1. Reflection

Creating the initial physical design of the database was not super difficult for us because we had a lot of practice throughout the semester. However, as we started to create the use cases, we discovered that our Library database was too simple to handle some of them. The most challenging part of this project was implementing the Java code and getting the UI to work. It would have been easier for us to implement the Java code if we had seen demos during class similar to the SQL query examples. We also had difficulties using the H drive; we could not figure out how to give access to everyone in the group, so only one person had access to the entire project. Additionally, we had trouble backing up our SQL code.